sohamalloc

A general-purpose memory allocator that is **safe to call from signal handlers**, hardened by default, and quick to give memory back.

Soham Basanwar

Project review · November 2026

C11 · drop-in malloc / free / new / delete via LD_PRELOAD · Linux x86-64, arm64, armhf, musl · Windows (soham_* API)



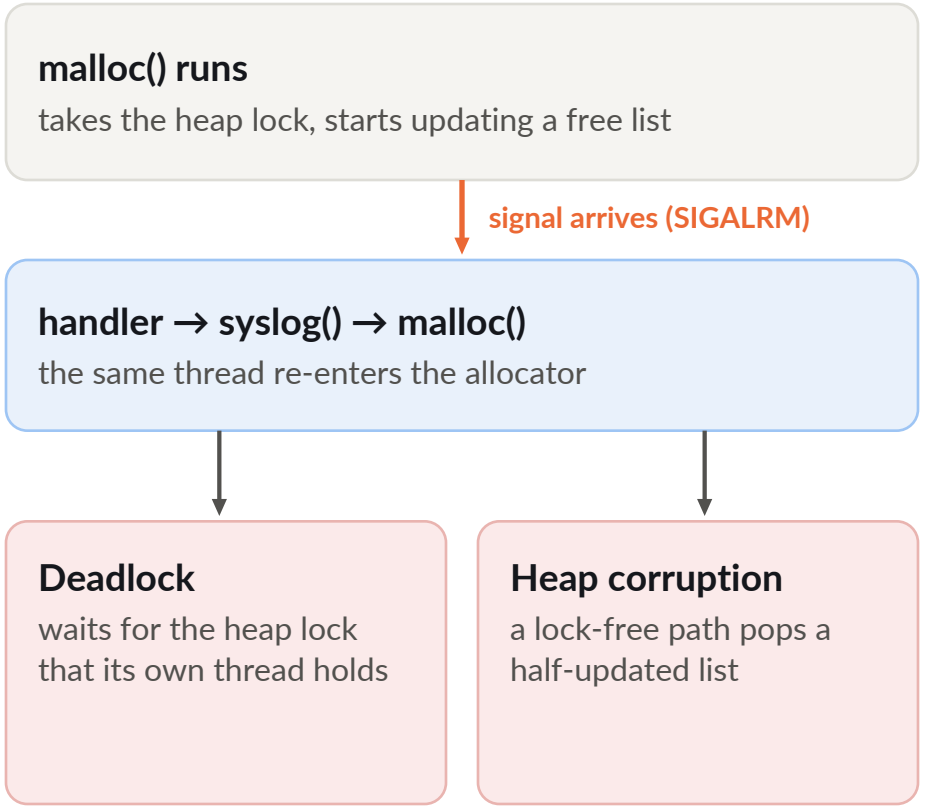
one 2 MiB hugeblock = 32 slices of 64 KiB: used · free and resident · free and zero · being returned to the kernel

MOTIVATION

Signal handlers call malloc, and allocators break

- A signal can interrupt a thread at **any instruction**, including inside `malloc`.
- POSIX: `malloc` and `free` are **not async-signal-safe**. A handler that calls them has undefined behaviour.
- Real software does it anyway, through `syslog`, logging, backtraces, C++ `new`:
  - **OpenSSH sshd, CVE-2024-6387 (“regreSSHion”)**: the SIGALRM handler reached malloc via syslog, giving remote code execution as root on glibc Linux.
  - Hangs reported in GCC, GNU make, Perl, g3log, a jemalloc user; a Red Hat note titled “permanent deadlock”.
- Current workarounds change the program (defer work to a safe point) or block signals around every allocation (two system calls per call).

HOW IT FAILS (ONE THREAD)



In our signal-storm test, all eight other allocators fail in 50–100% of trials.

THESIS AND RESULTS

Signal-safe malloc at competitive speed

A general-purpose, thread-caching allocator can make **the whole malloc API async-signal-safe** with no system call on the fast path, while staying as fast as the best allocators.

0 / 630 **signal-storm trials failed**

Final code; 95% CI 0–0.6%. The eight other allocators tested failed in 50–100% of their trials.

1.142 **time + memory score, tied first**

24 benchmarks, 5 allocators: mimalloc 1.142, glibc 1.276, jemalloc 1.344, tcmalloc 1.451 (lower is better).

50 MB **resident after freeing 1 GiB**

Without malloc_trim. glibc, jemalloc and mimalloc keep about 1 GiB; tcmalloc keeps 628 MB.

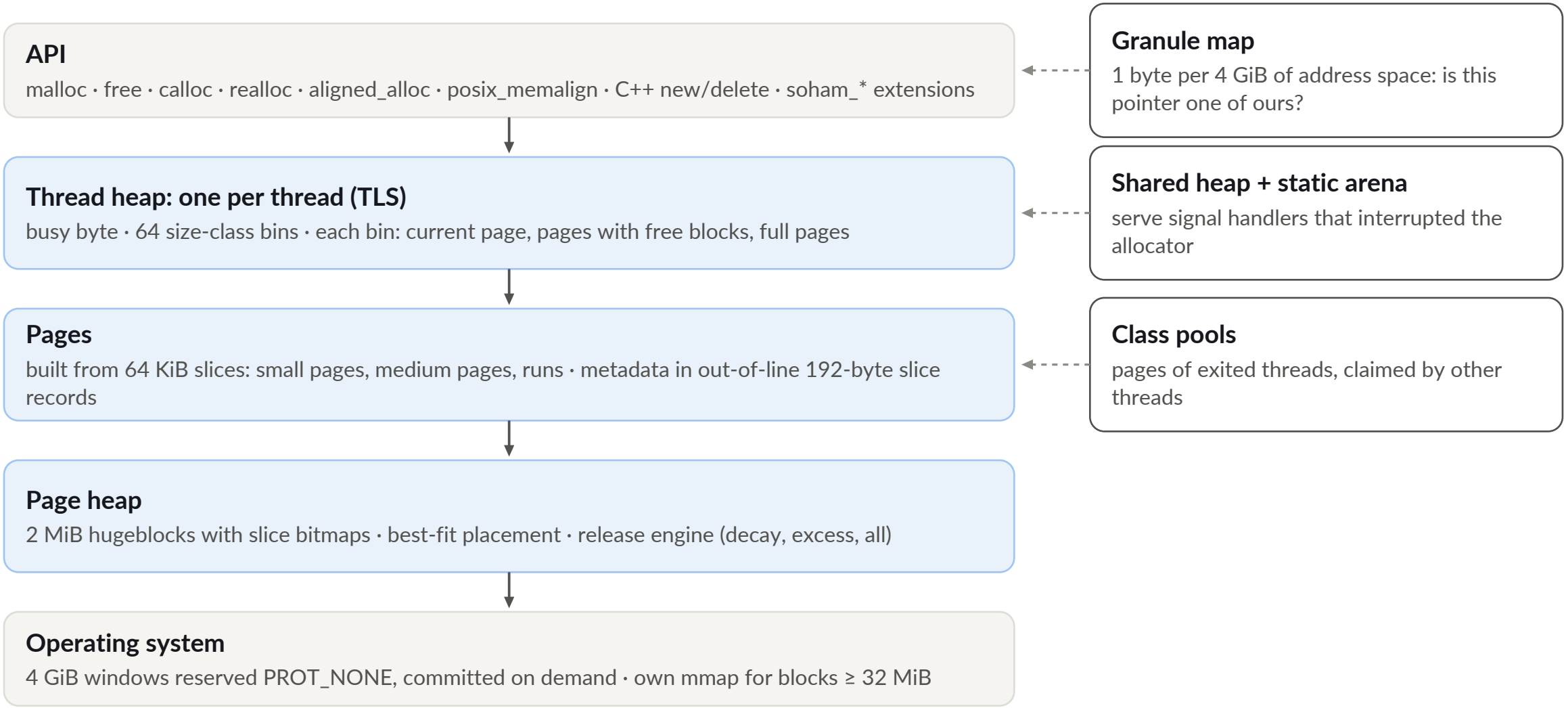
1–7% **cost of the default-on checks**

Encoded free lists and double/invalid-free detection, measured on allocation-heavy microbenchmarks.

Benchmarks: campaign B4 on a 4-vCPU VM, 3 repetitions, code of 2026-10-19. A tuned campaign with ≥10 repetitions on the final code is still to run.

DESIGN OVERVIEW

Architecture: five layers and three helpers



MEMORY LAYOUT

Address space: windows, hugeblocks, slices

ONE WINDOW: 4 GiB, aligned to 4 GiB, reserved PROT_NONE at creation



ONE HUGEBLOCK: 2 MiB = 32 SLICES OF 64 KiB



Pointer → metadata in O(1), no search

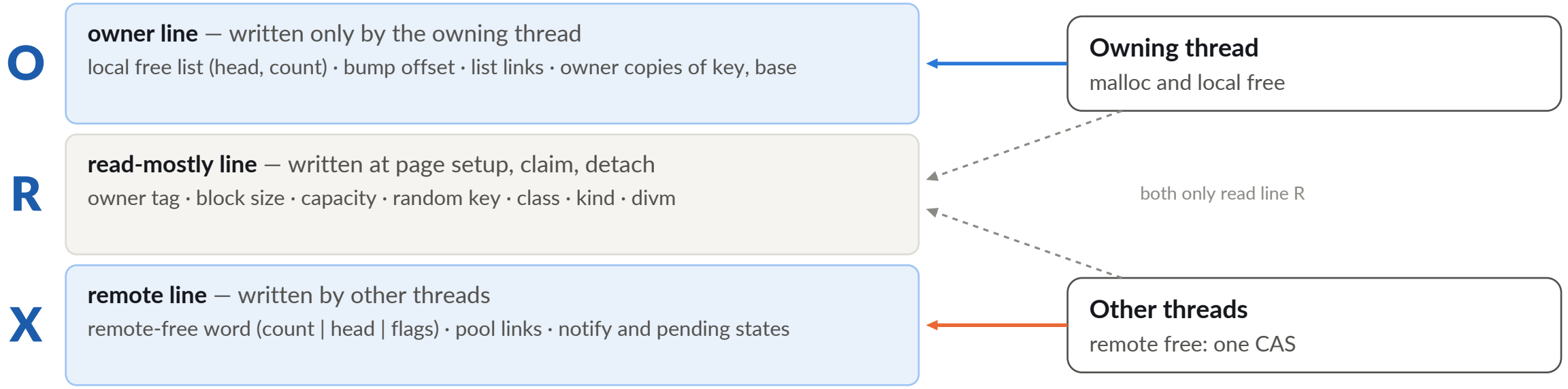
Granule map: one byte per 4 GiB of the 48-bit space (65,536 entries). Zero means “not ours”, so frees of stack, global or foreign pointers are caught.

Windows are reserved, not committed: data is committed in 2–64 MiB steps as the heap grows.

```
g   = gmap[(a >> 32) & 0xffff];          // ours?
rec = (a & ~(W-1)) + rec_off
    + ((a & (W-1)) >> 16) * 192;          // its record
```

MEMORY LAYOUT

Slice records: three cache lines, three writers

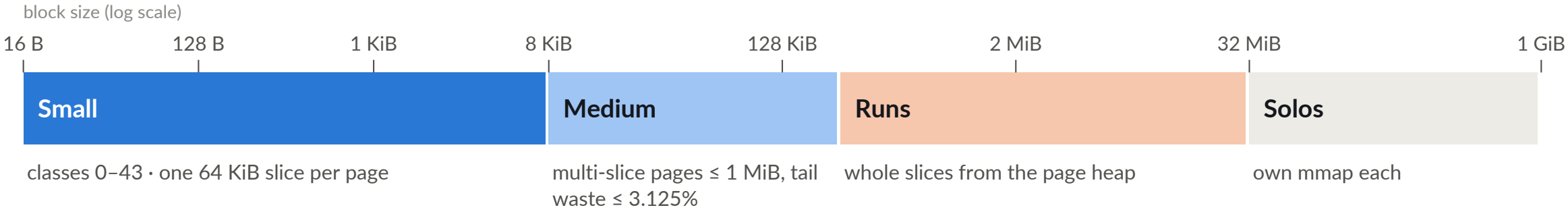


192 bytes = 3 × 64-byte cache lines, one record per 64 KiB slice, outside the data area

- The owner's malloc touches only line O and the block itself.
- A remote free reads R and writes X: it never touches O, so the owner's line does not bounce between cores.
- No allocator metadata sits next to user data; a heap overflow cannot reach it.

MEMORY LAYOUT

Size classes and the four ways a block is served



Range	Class spacing	Rounding loss per block
16 B – 128 B	16-byte steps (8 classes)	at most 15 bytes
128 B – 1 KiB	8 classes per doubling	under 12.5%
1 KiB – 256 KiB	4 classes per doubling	under 25%
> 256 KiB	page-heap runs, exact slices	under one 64 KiB slice

The 192 KiB and 256 KiB classes are served as runs. 64 classes in all; size → class is a table lookup up to 1 KiB and a few shifts above.

FAST PATHS

malloc fast path: no locks, no atomic read-modify-write

```
void *soham_malloc(size_t n) {
  tls_t *t = &t_tls;                      // initial-exec TLS
  if (t->busy == 0 && n <= FAST_MAX) {    // up to 128 KiB
    unsigned c = size_class(n);           // table lookup to 1 KiB
    t->busy = BUSY_IN;                    // a signal now sees "inside"
    rec_t *r = t->heap->bins[c].cur;      // current page of class c
    uintptr_t b = r->fhead;               // first free block (line 0)
    if (b) {
      uintptr_t nx = link_dec(r->key, *(uintptr_t *)b, b);
      if (nx && !exact_block(r, nx))      // must be a block boundary
        fatal("free-list link overwritten");
      *(uintptr_t *)b = 0;                // aids double-free detection
      r->fhead = nx;  r->fcnt--;
    }
    t->busy = 0;
    if (b) return (void *)b;
  }
  return malloc_slow(n);  // refill, large sizes, handler context
}
```

1. **One TLS byte decides**
   busy = 0: a normal call. Set: this call interrupted the allocator, so it takes the handler path.
2. **Class lookup**
   a table for ≤ 1 KiB, a few shifts above
3. **Pop the current page's list**
   owner-only cache line, plain loads and stores
4. **Validate the next link**
   decoded link must be an exact block of this page, or abort
5. **Clear the link word**
   makes double-free detection on free cheap

About 45 instructions in all (callgrind); mimalloc's is about 18. The difference buys the checks and the busy byte.

FAST PATHS

free fast path: is the pointer ours, and is the page mine?

```
void soham_free(void *p) {
  uintptr_t a = (uintptr_t)p;
  if (gmap[(a >> 32) & 0xffff] == GM_WINDOW   // one of our windows?
      && t_tls.busy == 0) {
    rec_t *r = rec_of(a);                     // address arithmetic
    t_tls.busy = BUSY_IN;
    if (r->head_delta) r = rec_head(r);       // multi-slice page
    if (r->owner == t_tls.tag) {              // my page: local free
      owned_free(t_tls.heap, r, a);           // checks, push on line 0
      t_tls.busy = 0;
      return;
    }
    t_tls.busy = 0;
  }
  free_slow(p);  // remote free (1 CAS), runs, solos, foreign pointers
}
```

Local free (my page)

Checks the exact block boundary, then one cheap test: does the block's first word decode to 0 or to an address in its page? Only then a cold path walks the lists and aborts on a double free.

Remote free (another thread's page)

One compare-and-swap on the page's remote line; the owner is notified only when it pays (next slides).

Not ours

Stack, global or foreign pointer: leaked with a warning (aborted in secure mode), never written to.

About 79 instructions on average (callgrind, glibc-simple); mimalloc's free is about 29.

SECURITY

Hardening that is on by default

Encoded free-list links

```
stored = bswap(next ^ key ^ (addr >> 12))
```

Per-page random key. A partial overwrite scrambles the high bits, the decoded link fails the boundary check, and the allocator aborts.

Exact block check, no division

```
divm = UINT64_MAX / bsize + 1;
ok   = off * divm < divm;  // off % bsize == 0
```

Lemire, Kaser and Kurz (2019). Applied to every decoded link and every free.

Invalid and double frees abort

- interior pointers, and non-adjacent double frees (`free(a); free(b); free(a)`)
- double frees into another thread's remote list, and into pages of exited threads
- foreign pointers are leaked with a warning, never written to

Cost: 1–7% on allocation-heavy microbenchmarks (glibc-simple 7%, cfrac 4%, alloc-test and espresso 1%).

Secure tier S1 (opt-in): shuffled block order, zeroing on free, abort on foreign pointers.

Honest limit: someone who can read free blocks can recover keys. Encoding raises the bar; it is not a guarantee.

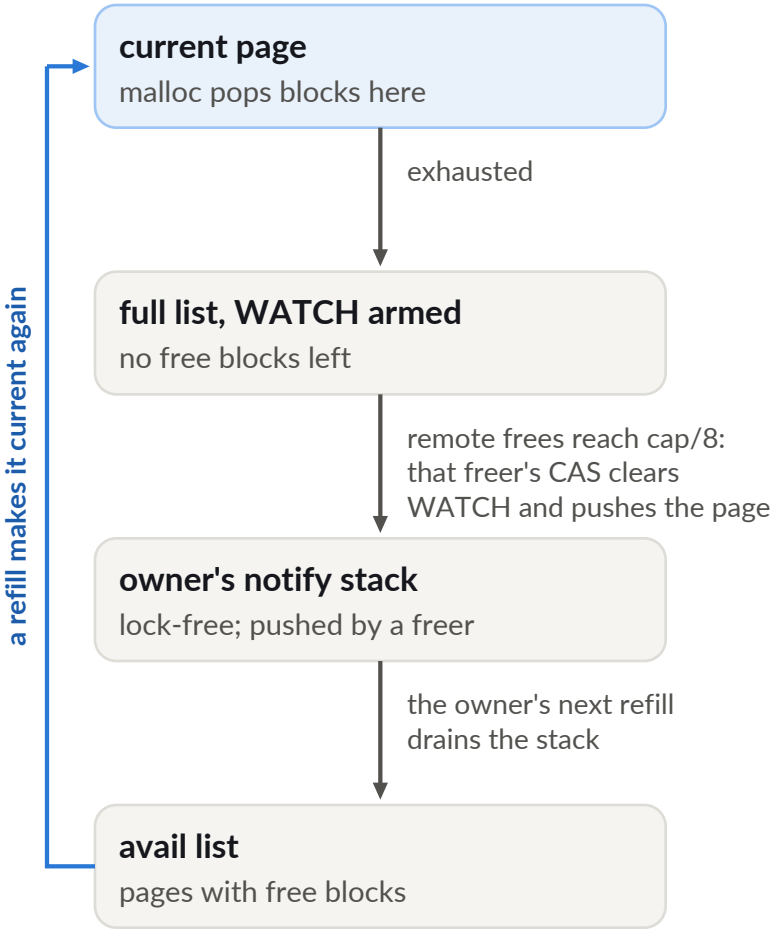
Cross-thread frees: one CAS, and the owner is told when it pays

```
do {
  old = page->xfree;                // count | head | flags (line X)
  *block = enc(old.head);           // link into the remote chain
  new = { old.count + 1, block, old.flags };
  if (WATCH && old.count + 1 >= cap/8)
    new.flags &= ~WATCH;            // this freer notifies the owner
} while (!CAS(&page->xfree, old, new));
```

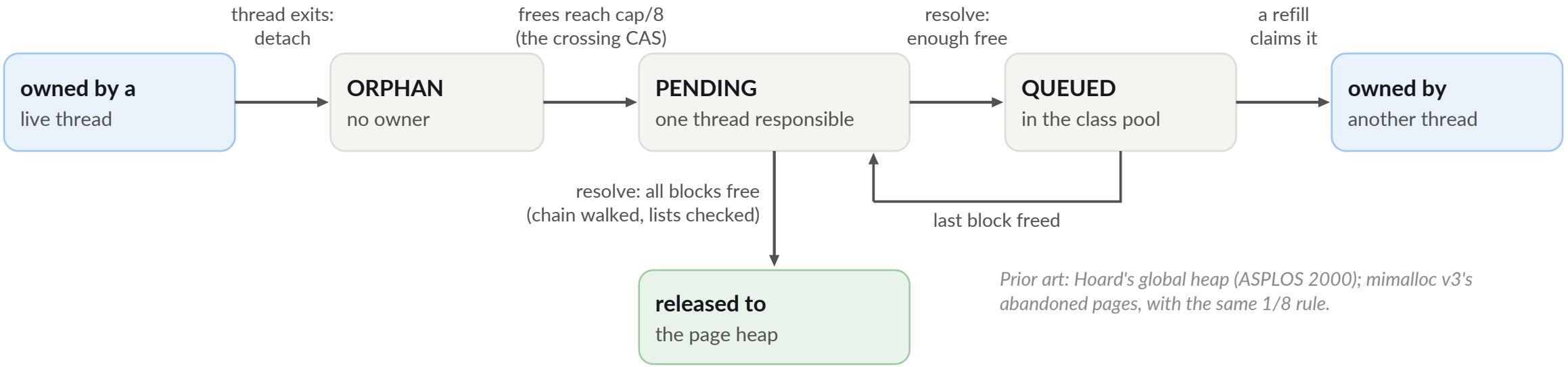
- **Checks on every remote free:** head == block (an immediate double free) and count > capacity.
- **What did not work first:** notifying the owner at the *first* free. Each refill then got a single block, and 28% of mallocs took the slow path (alloc-test).
- **Fix:** a threshold of cap/8, for remote and for local frees. Slow path 28% → 0.07%; alloc-test 2,228 → 1,966 ms. Cost: up to cap/8 − 1 idle blocks per full page.

Built on mimalloc's free-list sharding and delayed free (APLAS 2019); the threshold follows Hoard's emptiness hysteresis and mimalloc v3's 1/8 rule.

A PAGE ON ITS OWNER'S LISTS



Threads that exit leave no stranded memory



Prior art: Hoard's global heap (ASPLOS 2000); mimalloc v3's abandoned pages, with the same 1/8 rule.

- Every state change is a CAS on the page's remote-free word. The thread whose CAS makes the change becomes responsible, so no page is lost or handled twice.
- Detach clears the owner *before* publishing ORPHAN, so a later claimer's owner store can never be overwritten.
- Before a page is released or claimed, its remote chain is walked exactly and checked against its local list: a double free split across both lists aborts (review round 4).
- Signal handlers never resolve pages: they push them on a pending stack for the next slow path.

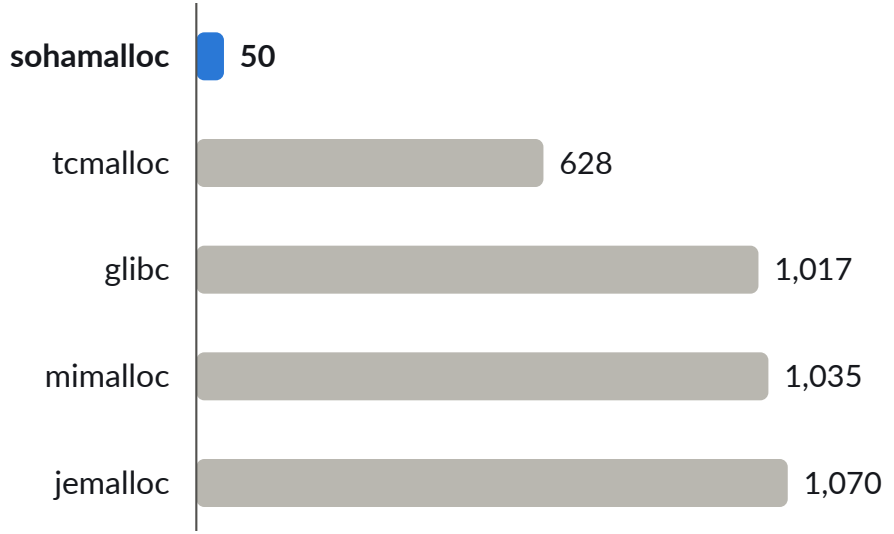
MEMORY RETURN

Page heap: hugepage-aware, and it gives memory back

- Each 2 MiB hugeblock keeps four slice bitmaps: **used, dirty** (free and resident), **zero, releasing**.
- Placement: best-fit up to 32 slices. Large runs come from per-window bitmaps of free (and resident) hugeblocks: a 4 MiB malloc + free went from 10 µs to 0.19 µs.
- Release modes: **decay** (dirty for ≥ 1 s), **excess** (free resident memory above a limit), **all** (malloc_trim).
- Limit = max(32 MiB, used/8, recent swing): a program that frees and re-allocates big buffers keeps what it is about to reuse.
- Release system calls run **outside the lock**: ranges are marked “releasing”, then one batched process_madvise covers up to 64 ranges.

Resident memory after freeing ≈ 1 GiB (MB)

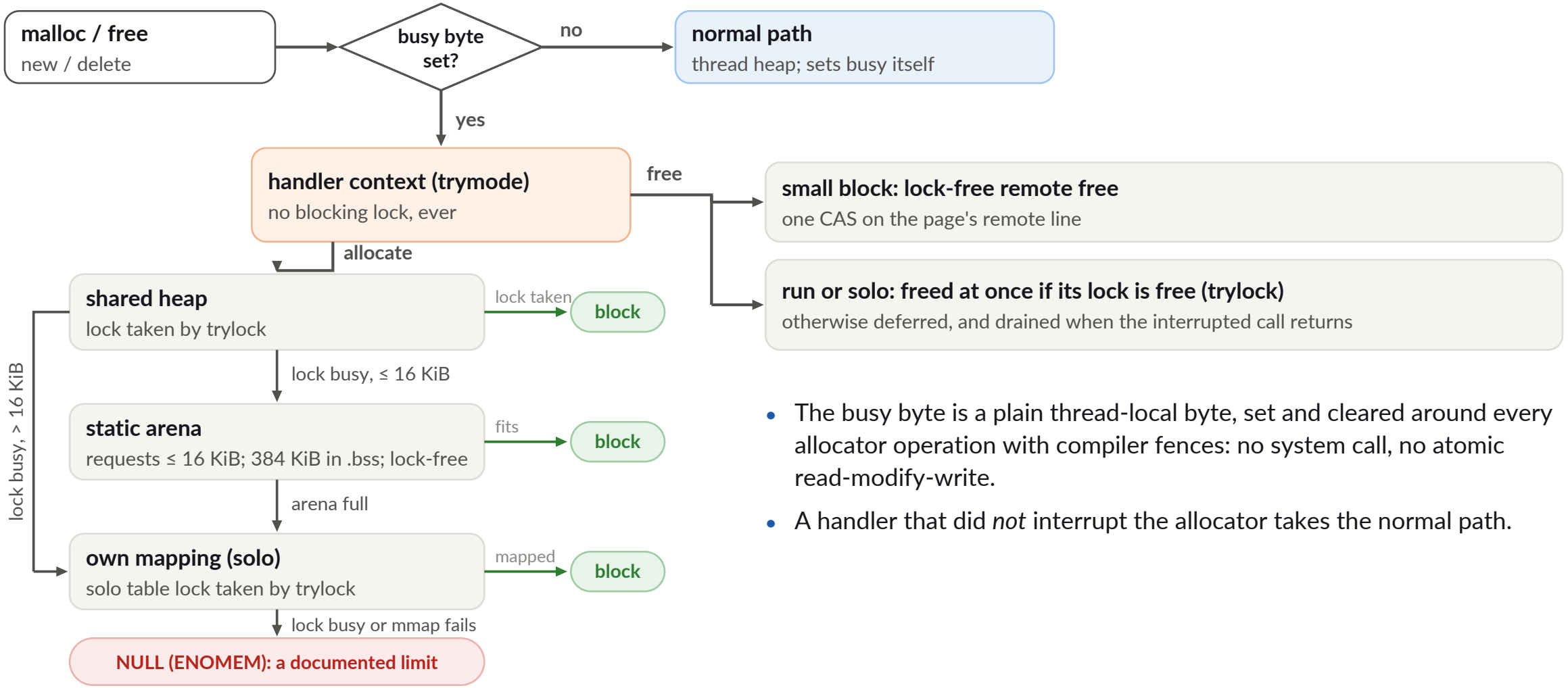
allocate ≈ 1 GiB, free all of it, measure RSS; no malloc_trim



After malloc_trim: glibc 9 MB and sohamalloc 14 MB; jemalloc, mimalloc and tcmalloc unchanged. Peak for all: 1,017–1,070 MB.

Prior art: TCMalloc Temeraire (OSDI 2021) and skip-subrelease; jemalloc's process_madvise batching (2025).

The busy byte routes a re-entrant call to a safe path



- The busy byte is a plain thread-local byte, set and cleared around every allocator operation with compiler fences: no system call, no atomic read-modify-write.
- A handler that did *not* interrupt the allocator takes the normal path.

SIGNAL SAFETY

Why it cannot deadlock or corrupt the heap

H1

One holder per heap

Only the thread running a heap changes its pages' owner words, so the local-free check owner == my tag cannot go stale.

H2

Busy byte around everything

Set around every operation that touches the thread's heap or takes any lock: fast and slow paths, init, thread attach and exit, stats, collect, check.

H3

No blocking lock in handler context

Every lock is a trylock; work that needs a real lock is skipped. Debug builds abort if H3 is ever violated.

Consequence

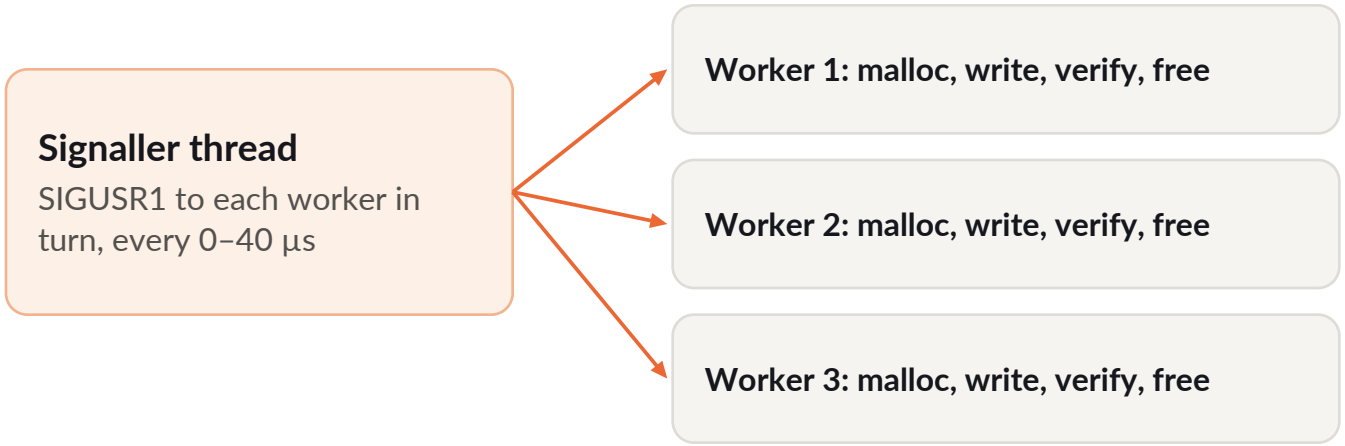
A handler never waits for a lock its own thread holds, and never waits for other threads either: a taken lock makes the request fall back. Lock-free, not wait-free.

fork() and initialisation

Fork handlers are registered at load; prepare finishes init and takes every lock in a fixed order (heap → shared → pool → page heap → solo → meta). Init is owned by a process id, so a child can finish an init that fork cut off. The busy byte is saved per thread across fork.

EVALUATION

Evaluation 1: signal storms (our tool, sigstorm)



1, 2 or 4 workers; the handler calls the allocator

Handler kinds

malloc · free · realloc · calloc · large (300 KB) · churn (bursts) · aligned (32 B – 64 KiB)

Controls

no signals; signals with a handler that does not allocate

Outcome of each 5 s trial

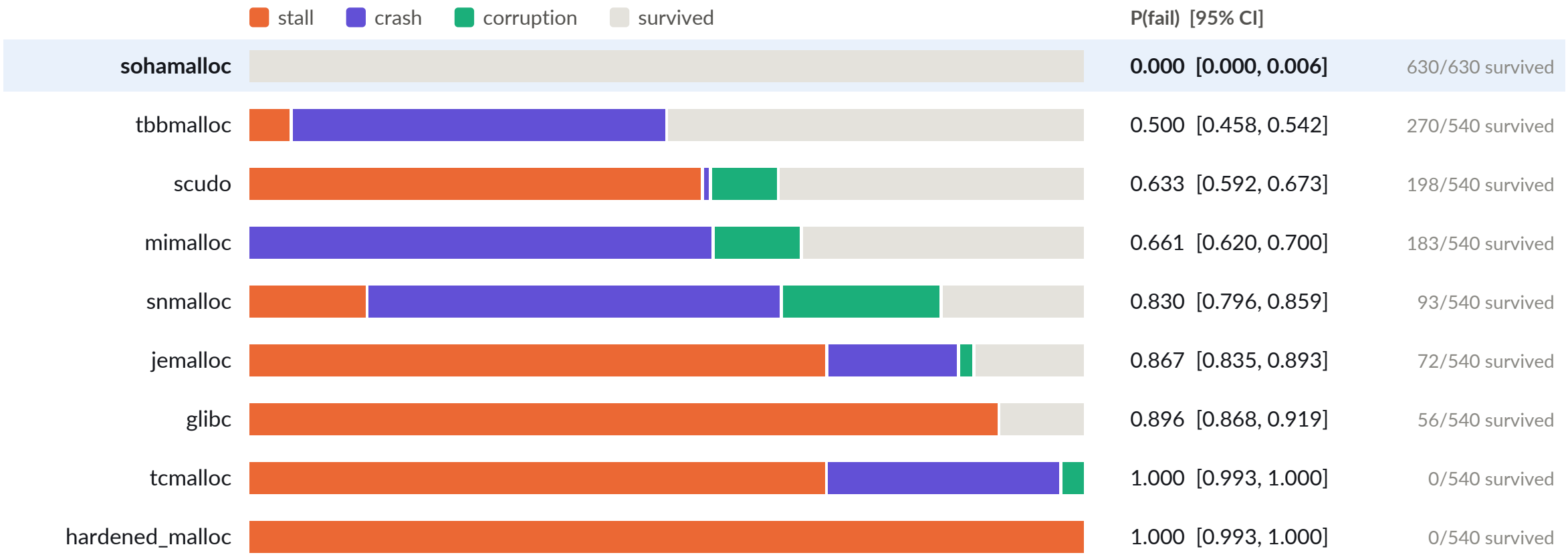
survived · **stall** (a thread makes no progress for 2 s) · **corruption** (contents changed, blocks overlap) · **crash**

Campaign	Allocators	Trials
Campaign 1 (2026-10-21)	9, incl. glibc, jemalloc, mimalloc, tcmalloc	6,480
aligned kind for the other eight	8	720
sohamalloc on the final code	1	810

30 trials per cell; 95% Wilson intervals; frozen library builds with sha256; one machine (Intel Core Ultra 9 185H, 22 CPUs).

EVALUATION

Results: only sohamalloc survives the storm



Others: campaign 1 (540 signal trials each). sohamalloc: final code d2add24, 630 signal trials, 140 million handler calls, 2.7 million of them inside an interrupted allocator call. Controls: 1,800 trials, 0 failures. With the aligned kind, the others failed 39–90 of 90 trials.

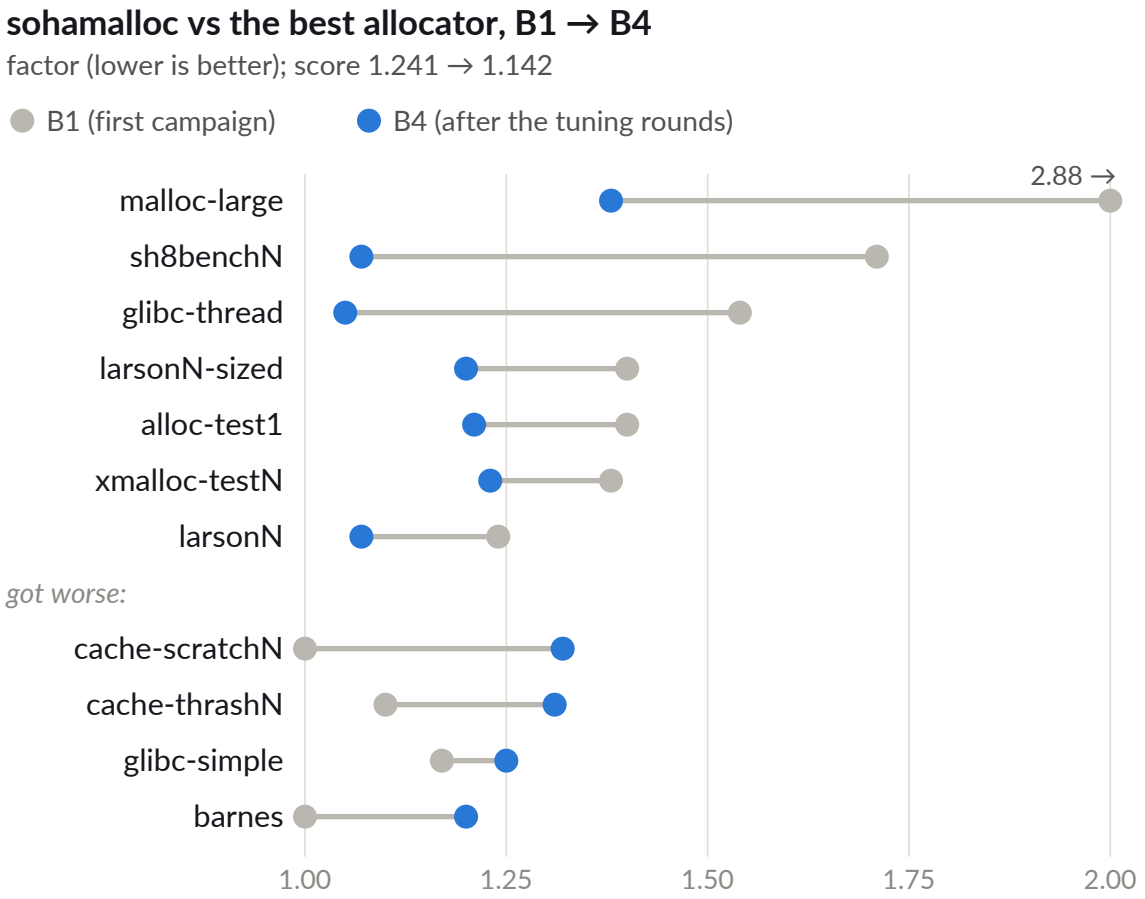
EVALUATION

Evaluation 2: speed and memory on 24 benchmarks

Allocator	Time	RSS	Score
sohamalloc	**1.124**	**1.162**	**1.142**
mimalloc 2.1.2	**1.063**	1.227	**1.142**
glibc 2.39	1.491	1.092	1.276
jemalloc 5.3.0	1.333	1.355	1.344
tcmalloc 2.15	1.370	1.537	1.451

Geometric means over 24 workloads (mimalloc-bench subset plus python, gcc, sqlite and redis) of time and peak RSS, each normalised to the best allocator per workload; score = geometric mean of the two. Lowest RSS among the two leaders; mimalloc is faster.

Caveats: 4-vCPU cloud VM, 3 repetitions, distribution builds of the other allocators, code of 2026-10-19. The tuned campaign is gate 4.



HONEST COMPARISON

Where other allocators are still better, and why

Where	Better	Why
Raw fast-path speed	mimalloc	~18 / 29 instructions for malloc / free against our ~45 / 79: we pay for link checks, double-free detection, the foreign-pointer test, the busy byte
Producer / consumer (larson)	mimalloc, 1.2×	remote frees, notification and chain validation cost more; open item
Random-order frees	mimalloc	alloc-test1 1.21×, xmalloc-testN 1.23×: the cap/8 threshold trades refills for idle blocks
barnes, cache-thrash/scratch	glibc, mimalloc	1.20–1.32×; runs of about 0.5 s near timer resolution, spread up to 117%: not yet explained
Tiny processes' memory	glibc	+1.6–3.2 MiB RSS from windows, records and the granule map
Tight RLIMIT_AS	glibc	at 128 MiB: 60 MiB usable against 100 MiB (reserved windows count)
Hardening depth	snmalloc, hardened_malloc	signed back-pointers, stronger isolation; our keys come from one stream per heap
Progress guarantee	Michael (2004), LRMalloc	fully lock-free; our handler path is lock-free but may return NULL when locks are held

VERIFICATION

How we know: tests, sanitizers, platforms, reviews

26/26 native	**26/26** debug assertions	**11/11** prefixed API
9/9 ThreadSanitizer	**9/9** AddressSanitizer	**9/9** UBSan
9/9 real programs (preload)	**26/26** aarch64 (qemu)	**26/26** armhf 32-bit (qemu)
25/25 musl	**9/9** Windows (mingw, wine)	**66/66** sigstorm, debug build

Latest full matrix, 2026-10-23, on the round-4 fixes. Skipped by design: 2 sanitizer cases, 1 C++ case on musl, 2 on Windows. Real programs: python3, gcc, g++, git, perl, sort, find, tar, bash.

Review round	Confirmed	Status
Round 2 (10-21)	2 critical, 1 major, 2 minor	fixed
Round 3 (10-21)	1 critical, 3 major, 2 minor	fixed; minors documented
Storm campaign 1	1 major (C1-1)	fixed
Round 4 (10-22/23)	2 critical, 4 major, 10 minor	fixed or documented
Round 5	—	not yet completed

Rule: every defect gets a regression test that fails on the old code and passes on the fix, run before any allocator commit.

Reviews: independent reviewer agents with separate lenses (signals, concurrency, lifecycle, API, portability), plus refuters that try to disprove each finding; a finding counts once reproduced.

RELATED WORK

What we build on, and what is new

Built on published ideas

- Free-list sharding, one-CAS remote frees, abandoned-page reclaim: mimalloc (APLAS 2019; v2, v3), Hoard (ASPLOS 2000), rpmalloc
- Address-to-metadata byte map: snmalloc (ISMM 2019)
- Hugepage-aware page heap, demand-based release: TCMalloc Temeraire (OSDI 2021)
- Link encoding: glibc Safe-Linking (2020), PartitionAlloc, mimalloc secure mode
- Batched process_madvise: jemalloc (2025)

What is new

- A **documented async-signal-safety guarantee for the whole malloc API** in a user-space, thread-caching, general-purpose allocator, with no system call on the fast path.
- Together with default-on hardening and prompt memory return, at mimalloc-class speed.
- Closest prior work: lock-free allocators (Michael, PLDI 2004; LRMalloc 2018); Linux kernel any-context allocators (bpf_mem_alloc; kmalloc_nolock, 2025); special-purpose pools (AO_malloc, folly, Abseil LowLevelAlloc).

Framing: an integration and experience paper with a strong evaluation, not one new mechanism.

HONEST LIMITS

Limits of the guarantee: stated, not hidden

1. **A handler's allocation can return NULL while memory is available**

   when the locks it needs are held and both fallbacks fail. Measured on the final code: 31 of 19.9 million large-block handler calls, 2 of 20.1 million churn calls, 0 for the other kinds.

2. **Frees of large blocks in a handler may be deferred**

   until the interrupted call returns.

3. **The handler must return normally**

   no siglongjmp or exception out of an interrupted allocator call.

4. **fork() from such a handler is not covered**

   POSIX.1-2024 no longer lists fork as async-signal-safe.

5. **The very first allocator call made from a handler**

   uses pthread and mmap functions that are not on POSIX's list (safe on glibc and musl in practice).

6. **Evidence, not proof**

   regression tests, the debug check of H3, adversarial reviews and storms. Storms alone missed a real deadlock (R2-1): model checking is future work.

STATUS

Status: the four gates before a paper

✓ **1. Every known bug fixed, each with a fail-before / pass-after test**
Met. Every confirmed critical and major defect has its test; the full matrix passes on d2add24.

◯ **2. A review round with no critical or major finding**
Open. Round 5 did not complete (usage limits); it must be relaunched.

✓ **3. Signal-storm campaign: ≥ 30 trials per cell, CIs, controls, limits**
Met. 0 of 630 signal trials failed on the final code (95% CI 0–0.6%); limits documented.

◯ **4. Tuned benchmark campaign: ≥ 10 repetitions with CIs**
Open. Not started: it needs a quiet machine tuned for benchmarking.

Next steps

- Review round 5; fix and repeat until a round is clean
- Measure the busy byte's cost (ablation, with CIs)
- Tuned benchmarks on the final code, with layout randomisation
- Close the larson gap; return idle threads' memory; keyed per-page keys
- Model-check the lock-free page protocols
- Then write the paper

METHOD

How the work was done

- **Directed by me:** the goals, the thesis, priorities and the four acceptance gates. Key choices were taken as explicit decisions and logged.
- **AI-assisted tooling did most of the hands-on work:** literature research, competing designs and their critique (a 101-fault critique, a red team), implementation, tests, benchmarks, and adversarial code reviews run as multi-agent workflows.
- **Trust comes from evidence, not from the tool:** every claim is backed by tests, raw data and before/after records in the repository; a review finding counts only once a reproducer confirms it.
- **Research log:** a timeline of every phase, a defect ledger with root causes and fixes, and a novelty analysis against 50 references.

DISCUSSION

Where I would value your advice

Venue and framing

An integration and experience paper (ISMM, or a systems venue)? What would make the signal-safety contribution stand out?

Evaluation

Which real-world workloads would convince reviewers: crash reporters, profilers, logging libraries, language runtimes?

Verification

Is model checking the lock-free page protocols (TLA+, GenMC) worth the time before submission?

Cost of safety

How best to isolate the busy byte's cost: ablation builds, hardware counters, layout-randomised runs?